./jiahe.json


```
{
  "type": "keystroke_dataset",
  "version": 3,
  "created_unix": 1772266319,
  "run_format": "native_key_events_v1",
  "score_threshold": 0.72,
  "feature_dim": 19,
  "num_enrollment_runs": 34,
  "num_test_runs": 10,
  "num_enrollment_raw_runs": 34,
  "num_test_raw_runs": 10,
  "enrollment_runs": [
    {
      "events": [
        {
          "type": "keydown",
          "timestamp_ms": 954118875.0,
          "keycode": 16,
          "keysym": "Shift_L",
          "char": ""
        },
        {
          "type": "keydown",
          "timestamp_ms": 954119031.0,
          "keycode": 87,
          "keysym": "W",
          "char": "W"
        },
        {
          "type": "keyup",
          "timestamp_ms": 954119140.0,
          "keycode": 16,
          "keysym": "Shift_L",
          "char": ""
        },
        {
          "type": "keyup",
          "timestamp_ms": 954119140.0,
          "keycode": 87,
          "keysym": "w",
          "char": "w"
        },
        {
          "type": "keydown",
```

```
  "timestamp_ms": 954119218.0,
  "keycode": 69,
  "keysym": "e",
  "char": "e"
},
{
  "type": "keydown",
  "timestamp_ms": 954119296.0,
  "keycode": 32,
  "keysym": "space",
  "char": " "
},
{
  "type": "keyup",
  "timestamp_ms": 954119343.0,
  "keycode": 69,
  "keysym": "e",
  "char": "e"
},
{
  "type": "keyup",
  "timestamp_ms": 954119406.0,
  "keycode": 32,
  "keysym": "space",
  "char": " "
},
{
  "type": "keydown",
  "timestamp_ms": 954119406.0,
  "keycode": 72,
  "keysym": "h",
  "char": "h"
},
{
  "type": "keydown",
  "timestamp_ms": 954119453.0,
  "keycode": 65,
  "keysym": "a",
  "char": "a"
},
{
  "type": "keydown",
  "timestamp_ms": 954119531.0,
  "keycode": 86,
  "keysym": "v",
  "char": "v"
```

```
},
{
  "type": "keyup",
  "timestamp_ms": 954119562.0,
  "keycode": 72,
  "keysym": "h",
  "char": "h"
},
{
  "type": "keyup",
  "timestamp_ms": 954119562.0,
  "keycode": 65,
  "keysym": "a",
  "char": "a"
},
{
  "type": "keyup",
  "timestamp_ms": 954119640.0,
  "keycode": 86,
  "keysym": "v",
  "char": "v"
},
{
  "type": "keydown",
  "timestamp_ms": 954119640.0,
  "keycode": 69,
  "keysym": "e",
  "char": "e"
},
{
  "type": "keydown",
  "timestamp_ms": 954119703.0,
  "keycode": 32,
  "keysym": "space",
  "char": " "
},
{
  "type": "keyup",
  "timestamp_ms": 954119750.0,
  "keycode": 69,
  "keysym": "e",
  "char": "e"
},
{
  "type": "keyup",
  "timestamp_ms": 954119812.0,
```

```
      "keycode": 32,
      "keysym": "space",
      "char": " "
    },
    {
      "type": "keydown",
      "timestamp_ms": 954119828.0,
      "keycode": 69,
      "keysym": "e",
      "char": "e"
    },
    {
      "type": "keydown",
      "timestamp_ms": 954119906.0,
```

./keystroke_app/init.py

```
from .app import App, main

__all__ = ["App", "main"]
```

./keystroke_app/app.py

```
from copy import deepcopy
import tkinter as tk
from pathlib import Path
from tkinter import filedialog, messagebox, ttk
from typing import Any, Dict, List, Optional

import numpy as np

from .capture import RunCapture, build_feature_vector_from_raw_run
from .config import (
    DATASET_VERSION,
    DEFAULT_DATASET_PATH,
    PROMPT_QUEUE_SIZE,
    SENTENCE_DATASET_PATH,
)
from .prompts import PromptQueue, load_sentence_bank
from .storage import (
    SessionData,
    load_session_data,
    merge_session_data_file,
    save_session_data,
```

```
)
from .verifier import Verifier


SENTENCE_BANK = load_sentence_bank(SENTENCE_DATASET_PATH)


class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Keystroke Dynamics (Sentence Dataset) - Same vs Different")
        self.geometry("980x680")

        self.phase = "idle"  # idle | enroll | test | done
        self.phase_target_runs: Optional[int] = None

        self.prompt_queue = PromptQueue(SENTENCE_BANK, PROMPT_QUEUE_SIZE)
        self.capture = RunCapture()
        self.verifier = Verifier()

        self.enroll_samples: List[np.ndarray] = []
        self.enroll_raw_runs: List[Dict[str, Any]] = []
        self.test_samples: List[np.ndarray] = []  # cumulative, persisted with dataset save
        self.test_raw_runs: List[Dict[str, Any]] = []
        self.test_run_samples: List[np.ndarray] = []  # current test window only
        self.test_run_raw_runs: List[Dict[str, Any]] = []  # current test window only

        self.latest_phase_name: Optional[str] = None  # "enroll" | "test"
        self.latest_phase_data: Optional[SessionData] = None

        self._build_ui()
        self._render_prompt_queue()
        self._set_status("Click 'Start Enrollment' or load a dataset.")
        self._update_progress_ui()
        self._update_runs_label()
        self._set_idle_controls()

    def _build_ui(self):
        frm = ttk.Frame(self, padding=12)
        frm.pack(fill="both", expand=True)

        ttk.Label(
            frm,
            text="Type the top sentence exactly (including capitals and punctuation):",
            font=("Segoe UI", 12, "bold"),
        ).pack(anchor="w")
```

```
self.target_box = tk.Text(frm, height=10, wrap="word", font=("Consolas", 12))
self.target_box.pack(fill="x", pady=(6, 10))
self.target_box.configure(state="disabled")

ttk.Label(frm, text="Typing area:", font=("Segoe UI", 11, "bold")).pack(anchor="w")
self.input_box = tk.Text(frm, height=7, wrap="word", font=("Consolas", 12))
self.input_box.pack(fill="both", expand=False, pady=(6, 10))
self.input_box.tag_configure("mismatch", foreground="#c1121f")
self.input_box.focus_set()

self.input_box.bind("<KeyPress>", self._on_key_press)
self.input_box.bind("<KeyRelease>", self._on_key_release)

ctrl = ttk.Frame(frm)
ctrl.pack(fill="x", pady=(4, 10))

ttk.Label(ctrl, text="Enroll sentences:").pack(side="left")
self.enroll_target_var = tk.StringVar(value="20")
self.spn_enroll_target = ttk.Spinbox(
    ctrl,
    from_=1,
    to=500,
    increment=1,
    width=5,
    textvariable=self.enroll_target_var,
)
self.spn_enroll_target.pack(side="left", padx=(6, 10))

self.btn_enroll = ttk.Button(ctrl, text="Start Enrollment", command=self.start_enrol
self.btn_enroll.pack(side="left")

ttk.Label(ctrl, text="Test sentences:").pack(side="left", padx=(12, 0))
self.test_target_var = tk.StringVar(value="10")
self.spn_test_target = ttk.Spinbox(
    ctrl,
    from_=1,
    to=500,
    increment=1,
    width=5,
    textvariable=self.test_target_var,
)
self.spn_test_target.pack(side="left", padx=(6, 10))

self.btn_test = ttk.Button(ctrl, text="Start Test", command=self.start_test, state='
self.btn_test.pack(side="left", padx=(8, 0))
```

```
        self.btn_save_dataset = ttk.Button(
            ctrl,
            text="Save Dataset...",
            command=self.save_dataset,
            state="disabled",
        )
        self.btn_save_dataset.pack(side="left", padx=(8, 0))

        self.btn_load_dataset = ttk.Button(
            ctrl,
            text="Load Dataset...",
            command=self.load_dataset,
        )
        self.btn_load_dataset.pack(side="left", padx=(8, 0))

        self.btn_merge_dataset = ttk.Button(
            ctrl,
            text="Merge Dataset...",
            command=self.merge_dataset,
            state="disabled",
        )
        self.btn_merge_dataset.pack(side="left", padx=(8, 0))

        self.btn_reset = ttk.Button(ctrl, text="Reset", command=self.reset_all)
        self.btn_reset.pack(side="left", padx=(8, 0))

        stats = ttk.Frame(frm)
        stats.pack(fill="x", pady=(6, 0))

        self.lbl_progress = ttk.Label(stats, text="Progress: --", font=("Segoe UI", 11, "bol
        self.lbl_progress.pack(side="left")

        self.lbl_runs = ttk.Label(stats, text="Runs: 0 enroll / 0 test", font=("Segoe UI", 1
        self.lbl_runs.pack(side="left", padx=(16, 0))

        self.lbl_status = ttk.Label(frm, text="", font=("Segoe UI", 11))
        self.lbl_status.pack(fill="x", pady=(10, 0))

        self.output = tk.Text(frm, height=10, wrap="word", font=("Consolas", 11))
        self.output.pack(fill="both", expand=True, pady=(10, 0))
        self.output.configure(state="disabled")

    def _log(self, msg: str):
        self.output.configure(state="normal")
        self.output.insert("end", msg + "\n")
```

```
        self.output.see("end")
        self.output.configure(state="disabled")

    def _set_status(self, msg: str):
        self.lbl_status.configure(text=msg)

    @staticmethod
    def _first_mismatch_index(typed: str, target: str) -> Optional[int]:
        limit = min(len(typed), len(target))
        for i in range(limit):
            if typed[i] != target[i]:
                return i
        if len(typed) > len(target):
            return len(target)
        return None

    def _update_input_mismatch_highlight(self, typed: str, target: str):
        self.input_box.tag_remove("mismatch", "1.0", "end")
        mismatch_idx = self._first_mismatch_index(typed, target)
        if mismatch_idx is None or mismatch_idx >= len(typed):
            return
        start = f"1.0+{mismatch_idx}c"
        end = f"1.0+{len(typed)}c"
        self.input_box.tag_add("mismatch", start, end)

    def _update_runs_label(self):
        self.lbl_runs.configure(text=f"Runs: {len(self.enroll_samples)} enroll / {len(self.t

    def _current_phase_run_count(self) -> int:
        if self.phase == "enroll":
            return len(self.enroll_samples)
        if self.phase == "test":
            return len(self.test_run_samples)
        return 0

    def _update_progress_ui(self):
        if self.phase in ("enroll", "test") and self.phase_target_runs is not None:
            self.lbl_progress.configure(text=f"Progress: {self._current_phase_run_count()}/+
        else:
            self.lbl_progress.configure(text="Progress: --")

    @staticmethod
    def _parse_target_runs(raw_value: str, label: str) -> int:
        try:
            target = int(raw_value)
        except Exception as ex:
```

```
        raise ValueError(f"{label} must be an integer.") from ex
    if target < 1:
        raise ValueError(f"{label} must be at least 1.")
    return target

def _has_session_data(self) -> bool:
    if self.latest_phase_data is None:
        return False
    return bool(self.latest_phase_data.enrollment_samples) or bool(self.latest_phase_dat

def _clear_latest_phase_snapshot(self):
    self.latest_phase_name = None
    self.latest_phase_data = None

def _set_latest_phase_snapshot(self, phase_name: str):
    reference = None
    if self.verifier.reference is not None:
        reference = np.array(self.verifier.reference, dtype=np.float32, copy=True)

    if phase_name == "enroll":
        data = SessionData(
            enrollment_samples=[np.array(s, dtype=np.float32, copy=True) for s in self.e
            test_samples=[],
            enrollment_raw_runs=[deepcopy(run) for run in self.enroll_raw_runs],
            test_raw_runs=[],
            reference=reference,
            score_threshold=float(self.verifier.score_threshold),
        )
    elif phase_name == "test":
        data = SessionData(
            enrollment_samples=[],
            test_samples=[np.array(s, dtype=np.float32, copy=True) for s in self.test_ru
            enrollment_raw_runs=[],
            test_raw_runs=[deepcopy(run) for run in self.test_run_raw_runs],
            reference=reference,
            score_threshold=float(self.verifier.score_threshold),
        )
    else:
        raise ValueError(f"Unsupported phase snapshot: {phase_name}")

    self.latest_phase_name = phase_name
    self.latest_phase_data = data

def _set_running_controls(self):
    self.btn_enroll.configure(state="disabled")
    self.btn_test.configure(state="disabled")
```

```
        self.spn_enroll_target.configure(state="disabled")
        self.spn_test_target.configure(state="disabled")
        self.btn_save_dataset.configure(state="disabled")
        self.btn_load_dataset.configure(state="disabled")
        self.btn_merge_dataset.configure(state="disabled")
        self.btn_reset.configure(state="disabled")

    def _set_idle_controls(self):
        self.btn_enroll.configure(state="normal")
        self.btn_test.configure(state="normal" if self.verifier.has_reference() else "disabl
        self.spn_enroll_target.configure(state="normal")
        self.spn_test_target.configure(state="normal")
        has_data = self._has_session_data()
        self.btn_save_dataset.configure(state="normal" if has_data else "disabled")
        self.btn_load_dataset.configure(state="normal")
        self.btn_merge_dataset.configure(state="normal" if has_data else "disabled")
        self.btn_reset.configure(state="normal")

    def _render_prompt_queue(self):
        self.target_box.configure(state="normal")
        self.target_box.delete("1.0", "end")
        self.target_box.insert("1.0", self.prompt_queue.as_text())
        self.target_box.configure(state="disabled")

    def _reset_prompt_queue(self):
        self.prompt_queue.reset()
        self._render_prompt_queue()

    def _clear_input(self):
        self.input_box.delete("1.0", "end")
        self.input_box.tag_remove("mismatch", "1.0", "end")
        self.capture.reset()

    def _apply_session_data(self, data: SessionData):
        self.enroll_samples = [np.array(s, dtype=np.float32, copy=True) for s in data.enroll
        self.test_samples = [np.array(s, dtype=np.float32, copy=True) for s in data.test_sam
        self.enroll_raw_runs = [deepcopy(run) for run in data.enrollment_raw_runs]
        self.test_raw_runs = [deepcopy(run) for run in data.test_raw_runs]
        self.test_run_samples.clear()
        self.test_run_raw_runs.clear()

        if data.reference is not None:
            self.verifier.fit(data.reference)
            self.verifier.score_threshold = float(data.score_threshold)
        else:
            self.verifier.clear()
```

```
            self.verifier.score_threshold = float(data.score_threshold)

        self._update_runs_label()
        self._update_progress_ui()
        self._set_idle_controls()

    def reset_all(self):
        self.phase = "idle"
        self.phase_target_runs = None
        self.enroll_samples.clear()
        self.enroll_raw_runs.clear()
        self.test_samples.clear()
        self.test_raw_runs.clear()
        self.test_run_samples.clear()
        self.test_run_raw_runs.clear()
        self._clear_latest_phase_snapshot()
        self.capture.reset()
        self.verifier.clear()
        self._reset_prompt_queue()
        self._clear_input()
        self._update_runs_label()
        self._update_progress_ui()
        self._set_idle_controls()
        self._set_status("Reset. Click 'Start Enrollment' or load a dataset.")
        self._log("\n[Reset]\n")

    def load_dataset(self):
        if self.phase in ("enroll", "test"):
            messagebox.showwarning("Busy", "Stop the current run before loading a dataset.")
            return

        path_str = filedialog.askopenfilename(
            title="Load Dataset",
            initialdir=str(DEFAULT_DATASET_PATH.parent),
            filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
        )
        if not path_str:
            return

        path = Path(path_str)
        try:
            data = load_session_data(path)
        except Exception as ex:
            messagebox.showerror("Load failed", f"Could not load dataset:\n{ex}")
            return
```

```
        self._clear_latest_phase_snapshot()
        self._apply_session_data(data)
        self._reset_prompt_queue()
        self._clear_input()
        self._log(f"Loaded dataset from: {path}")
        self._set_status(f"Dataset loaded: {path.name}. Save/Merge now use next completed ph

    def save_dataset(self):
        if not self._has_session_data():
            messagebox.showwarning("No data", "No recent phase data is available to save. Co
            return

        path_str = filedialog.asksaveasfilename(
            title="Save Dataset",
            defaultextension=".json",
            initialdir=str(DEFAULT_DATASET_PATH.parent),
            initialfile=DEFAULT_DATASET_PATH.name,
            filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
        )
        if not path_str:
            return

        path = Path(path_str)
        try:
            assert self.latest_phase_data is not None
            save_session_data(path, self.latest_phase_data, DATASET_VERSION)
        except Exception as ex:
            messagebox.showerror("Save failed", f"Could not save dataset:\n{ex}")
            return

        phase_label = self.latest_phase_name if self.latest_phase_name is not None else "pha
        self._log(f"Saved latest {phase_label} dataset to: {path}")
        self._set_status(f"Dataset saved: {path.name}")
        self._set_idle_controls()

    def merge_dataset(self):
        if not self._has_session_data():
            messagebox.showwarning("No data", "No recent phase data is available to merge. C
            return

        path_str = filedialog.askopenfilename(
            title="Select Dataset JSON to Merge Into",
            initialdir=str(DEFAULT_DATASET_PATH.parent),
            filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
        )
        if not path_str:
```

```
            return

        path = Path(path_str)
        try:
            assert self.latest_phase_data is not None
            merged = merge_session_data_file(path, self.latest_phase_data, DATASET_VERSION)
            self._apply_session_data(merged)
        except Exception as ex:
            messagebox.showerror("Merge failed", f"Could not merge dataset:\n{ex}")
            return

        phase_label = self.latest_phase_name if self.latest_phase_name is not None else "pha
        self._log(f"Merged latest {phase_label} dataset into: {path}")
        self._set_status(f"Merged into dataset: {path.name}")

    def start_enroll(self):
        try:
            target_runs = self._parse_target_runs(self.enroll_target_var.get(), "Enroll sent
        except Exception as ex:
            messagebox.showwarning("Invalid setting", str(ex))
            return

        self.reset_all()
        self.phase = "enroll"
        self.phase_target_runs = target_runs
        self._reset_prompt_queue()
        self._set_running_controls()
        self._update_progress_ui()
        self._set_status(f"ENROLLMENT: Type {target_runs} sentence(s) exactly.")
        self._log(f"[Enrollment started] target runs = {target_runs}")

    def start_test(self):
        if self.phase not in ("idle", "done"):
            return
        if not self.verifier.has_reference():
            messagebox.showwarning("Not enough enrollment", "Enroll first or load a dataset
            return

        try:
            target_runs = self._parse_target_runs(self.test_target_var.get(), "Test sentence
        except Exception as ex:
            messagebox.showwarning("Invalid setting", str(ex))
            return

        self.phase = "test"
        self.phase_target_runs = target_runs
```

```
        self.test_run_samples.clear()
        self.test_run_raw_runs.clear()
        self._reset_prompt_queue()
        self._set_running_controls()
        self._update_progress_ui()
        self._set_status(f"TEST: Type {target_runs} sentence(s) exactly; completed lines wil
        self._log(f"\n[Test started] target runs = {target_runs}")
        self._clear_input()

    def _finish_phase(self):
        if self.phase == "enroll":
            self._log(f"[Enrollment finished] accepted runs = {len(self.enroll_samples)}")
            if len(self.enroll_samples) < 1:
                self._log("No accepted enrollment run captured. Try again.")
                self._set_status("Enrollment needs 1 completed run. Click 'Start Enrollment'
                self.phase = "idle"
                self.phase_target_runs = None
                self._set_idle_controls()
                self._update_progress_ui()
                return

            X = np.stack(self.enroll_samples, axis=0)
            reference = X.mean(axis=0).astype(np.float32)
            self.verifier.fit(reference)
            self._set_latest_phase_snapshot("enroll")
            self._log(f"Enrollment reference built from {len(self.enroll_samples)} run(s).")
            self._set_status("Enrollment complete. Save/Merge now use this enrollment snapsh
            self.phase = "idle"
            self._clear_input()
            self._set_idle_controls()

        elif self.phase == "test":
            self._log(f"[Test finished] accepted runs = {len(self.test_run_samples)}")
            if len(self.test_run_samples) < 1:
                self._log("No test runs captured. Try again and complete the prompt.")
                self._set_status("No test runs captured. Click 'Start Test' again and type c
                self.phase = "idle"
                self.phase_target_runs = None
                self._set_idle_controls()
                self._update_progress_ui()
                return

            Xtest = np.stack(self.test_run_samples, axis=0)
            try:
                scores, inlier = self.verifier.score(Xtest)
            except Exception as ex:
```

```
                self._log(f"Scoring failed: {ex}")
                self._set_status("Scoring failed due to reference/data mismatch. Re-enroll c
                self.phase = "idle"
                self.phase_target_runs = None
                self._set_idle_controls()
                self._update_progress_ui()
                return

            inlier_frac = float(inlier.mean())
            avg_score = float(scores.mean())

            self._log("\n--- RESULT ---")
            self._log(f"Avg decision score: {avg_score:.4f} (higher = more like enrolled use
            self._log(f"Inlier fraction:    {inlier_frac:.2%} (runs classified as 'User A-li

            same = inlier_frac >= 0.60
            self._log("VERDICT: " + ("SAME PERSON (likely)" if same else "DIFFERENT PERSON (

            self._set_latest_phase_snapshot("test")
            self._set_status("Test done. Save/Merge now use this test snapshot.")
            self.phase = "idle"
            self._set_idle_controls()

        self.phase_target_runs = None
        self._update_progress_ui()
        self._update_runs_label()

    def _maybe_accept_run(self):
        typed = self.input_box.get("1.0", "end-1c")
        target = self.prompt_queue.current()
        self._update_input_mismatch_highlight(typed, target)

        if typed == target:
            raw_run = self.capture.build_raw_run()
            if raw_run is None:
                self._log("Run matched text but no usable timing data was captured; avoid pa
            else:
                feat = build_feature_vector_from_raw_run(raw_run)
                if self.phase == "enroll":
                    self.enroll_samples.append(feat)
                    self.enroll_raw_runs.append(deepcopy(raw_run))
                    self._log(f"Accepted ENROLL run #{len(self.enroll_samples)}")
                elif self.phase == "test":
                    if self.verifier.reference is not None and feat.shape[0] != self.verifie
                        self._log("Rejected TEST run: feature dimension mismatch with loaded
                    else:
```

```
                    self.test_run_samples.append(feat)
                    self.test_samples.append(feat)
                    self.test_run_raw_runs.append(deepcopy(raw_run))
                    self.test_raw_runs.append(deepcopy(raw_run))
                    self._log(
                        f"Accepted TEST run #{len(self.test_run_samples)} "
                        f"(total saved tests: {len(self.test_samples)})"
                    )

                self._update_runs_label()
                self._update_progress_ui()

            # Consume top line and append a new sentence at the bottom.
            self.prompt_queue.advance()
            self._render_prompt_queue()
            self._clear_input()
            if self.phase_target_runs is not None and self._current_phase_run_count() >= sel
                self._finish_phase()
            return

        if len(typed) > len(target) + 5:
            self._set_status("You overshot the prompt. Use Reset or backspace; aim to match

    def _on_key_press(self, event):
        if self.phase not in ("enroll", "test"):
            return

        # Capture all keydown events (including modifiers) for raw dataset logging.
        self.capture.on_key_press(event)

        if event.keysym in {"Shift_L", "Shift_R", "Control_L", "Control_R", "Alt_L", "Alt_R'
            return

        if event.keysym in {"Delete", "Left", "Right", "Up", "Down", "Home", "End", "Return'
            return "break"

        ch = event.char
        if ch == "":
            return "break"

        # Force append-at-end behavior even if the caret was moved manually.
        self.input_box.tag_remove("sel", "1.0", "end")
        self.input_box.mark_set("insert", "end-1c")
        self.after(1, self._maybe_accept_run)

    def _on_key_release(self, event):
```

```
        if self.phase not in ("enroll", "test"):
            return
        self.capture.on_key_release(event)


def main():
    app = App()
    app.mainloop()
```

./keystroke_app/capture.py

```
import time
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

import numpy as np

RAW_RUN_FORMAT = "native_key_events_v1"


@dataclass
class KeyEventRec:
    kind: str
    timestamp_ms: float
    keycode: int
    keysym: str
    char: str


class RunCapture:
    """
    Captures native keydown/keyup keyboard events for one run.
    We capture all keydown/keyup events; text features are derived from printable keydowns.
    """

    def __init__(self):
        self.events: List[KeyEventRec] = []
        self.active_down_counts: Dict[int, int] = {}

    def reset(self):
        self.events.clear()
        self.active_down_counts.clear()

    @staticmethod
    def _event_timestamp_ms(event) -> float:
```

```
    native_ts = getattr(event, "time", None)
    if isinstance(native_ts, (int, float)):
        return float(native_ts)
    return float(time.time() * 1000.0)

def on_key_press(self, event):
    ch = str(getattr(event, "char", ""))
    keycode = int(event.keycode)
    self.events.append(
        KeyEventRec(
            kind="keydown",
            timestamp_ms=self._event_timestamp_ms(event),
            keycode=keycode,
            keysym=str(getattr(event, "keysym", "")),
            char=ch,
        )
    )
    self.active_down_counts[keycode] = int(self.active_down_counts.get(keycode, 0) + 1)

def on_key_release(self, event):
    keycode = int(event.keycode)
    if int(self.active_down_counts.get(keycode, 0)) <= 0:
        return
    self.events.append(
        KeyEventRec(
            kind="keyup",
            timestamp_ms=self._event_timestamp_ms(event),
            keycode=keycode,
            keysym=str(getattr(event, "keysym", "")),
            char=str(getattr(event, "char", "")),
        )
    )
    remaining = int(self.active_down_counts.get(keycode, 0)) - 1
    if remaining > 0:
        self.active_down_counts[keycode] = remaining
    else:
        self.active_down_counts.pop(keycode, None)

@staticmethod
def _timing_stats(values: np.ndarray) -> np.ndarray:
    if values.size == 0:
        return np.zeros(6, dtype=np.float32)
    q10, q50, q90 = np.quantile(values, [0.10, 0.50, 0.90])
    return np.array(
        [
            float(values.mean()),
```

```
            float(values.std()),
            float(q10),
            float(q50),
            float(q90),
            float(np.max(values)),
        ],
        dtype=np.float32,
    )

@classmethod
def _collect_run_stream_from_events(cls, events: List[Dict[str, Any]]) -> Optional[Tuple
    down_stack: Dict[int, List[int]] = {}
    chars_raw: List[str] = []
    down_times_s: List[float] = []
    dwell_raw: List[Optional[float]] = []

    for ev in events:
        kind = str(ev["type"])
        keycode = int(ev["keycode"])
        t_s = float(ev["timestamp_ms"]) / 1000.0

        if kind == "keydown":
            ch = str(ev.get("char", ""))
            is_text_key = ch == " " or (ch != "" and ch.isprintable())
            if not is_text_key:
                continue

            chars_raw.append(ch)
            down_times_s.append(t_s)
            dwell_raw.append(None)
            idx = len(chars_raw) - 1
            down_stack.setdefault(keycode, []).append(idx)
            continue

        if kind == "keyup":
            stack = down_stack.get(keycode)
            if not stack:
                continue
            idx = stack.pop()
            dwell = t_s - down_times_s[idx]
            if 0.005 <= dwell <= 2.0:
                dwell_raw[idx] = dwell

    dwell_list: List[float] = []
    flight_list: List[float] = []
    chars: List[str] = []
```

```
        if not down_times_s:
            return None

        prev_down: Optional[float] = None
        flights_raw: List[float] = []
        for t_down in down_times_s:
            if prev_down is None:
                flights_raw.append(0.0)
            else:
                flights_raw.append(max(0.0, t_down - prev_down))
            prev_down = t_down

        last_was_space = False
        for i, ch in enumerate(chars_raw):
            if ch.isspace():
                if last_was_space:
                    continue
                ch = " "
                last_was_space = True
            else:
                last_was_space = False

            d = dwell_raw[i] if i < len(dwell_raw) and dwell_raw[i] is not None else 0.0
            f = flights_raw[i] if i < len(flights_raw) else 0.0
            dwell_list.append(d)
            flight_list.append(f)
            chars.append(ch)

        if not dwell_list:
            return None

        dwell = np.array(dwell_list, dtype=np.float32)
        flight = np.array(flight_list, dtype=np.float32)

        dwell = np.clip(dwell, 0.0, 2.0)
        flight = np.clip(flight, 0.0, 2.0)
        return chars, dwell, flight

    def _collect_run_stream(self) -> Optional[Tuple[List[str], np.ndarray, np.ndarray]]:
        raw_run = self.build_raw_run()
        if raw_run is None:
            return None
        return self._collect_run_stream_from_events(raw_run["events"])

    @classmethod
```

```
def _feature_vector_from_parts(
    cls,
    chars: List[str],
    dwell: np.ndarray,
    flight: np.ndarray,
) -> np.ndarray:
    dwell_stats = cls._timing_stats(dwell)
    flight_stats = cls._timing_stats(flight)
    total_chars = float(len(chars))
    uppercase_ratio = float(sum(c.isalpha() and c.isupper() for c in chars) / max(1.0, t
    punctuation_ratio = float(sum(c in ".,;:!?" for c in chars) / max(1.0, total_chars))
    space_ratio = float(sum(c == " " for c in chars) / max(1.0, total_chars))
    long_pause_ratio = float((flight > 0.35).mean()) if flight.size else 0.0
    total_dwell = float(dwell.sum())
    total_flight = float(flight.sum())

    feat = np.concatenate(
        [
            np.log1p(dwell_stats),
            np.log1p(flight_stats),
            np.array(
                [
                    np.log1p(total_chars),
                    uppercase_ratio,
                    punctuation_ratio,
                    space_ratio,
                    long_pause_ratio,
                    np.log1p(total_dwell),
                    np.log1p(total_flight),
                ],
                dtype=np.float32,
            ),
        ],
        axis=0,
    ).astype(np.float32)
    return feat

def build_raw_run(self) -> Optional[Dict[str, Any]]:
    if not self.events:
        return None

    events = [
        {
            "type": ev.kind,
            "timestamp_ms": float(ev.timestamp_ms),
            "keycode": int(ev.keycode),
```

```
                "keysym": str(ev.keysym),
                "char": str(ev.char),
            }
            for ev in self.events
        ]
        return {
            "events": events,
        }

    def build_feature_vector(self) -> Optional[np.ndarray]:
        stream = self._collect_run_stream()
        if stream is None:
            return None
        chars, dwell, flight = stream
        return self._feature_vector_from_parts(chars, dwell, flight)


def normalize_raw_run(raw_run: Dict[str, Any]) -> Dict[str, Any]:
    if not isinstance(raw_run, dict):
        raise ValueError("Raw run must be an object.")

    events = raw_run.get("events")
    if not isinstance(events, list):
        raise ValueError("Raw run must include an 'events' list.")
    if not events:
        raise ValueError("Raw run must include at least one keyboard event.")

    out_events: List[Dict[str, Any]] = []
    last_ts: Optional[float] = None

    for i, event in enumerate(events):
        if not isinstance(event, dict):
            raise ValueError(f"events[{i}] must be an object.")

        kind = str(event.get("type", ""))
        if kind not in {"keydown", "keyup"}:
            raise ValueError(f"events[{i}].type must be 'keydown' or 'keyup'.")

        timestamp_ms_raw = event.get("timestamp_ms")
        if not isinstance(timestamp_ms_raw, (int, float)):
            raise ValueError(f"events[{i}].timestamp_ms must be numeric.")
        timestamp_ms = float(timestamp_ms_raw)
        if timestamp_ms < 0.0:
            raise ValueError(f"events[{i}].timestamp_ms must be >= 0.")
        if last_ts is not None and timestamp_ms < last_ts:
            raise ValueError("Event timestamps must be non-decreasing within a run.")
```

```
        last_ts = timestamp_ms

        keycode_raw = event.get("keycode")
        if not isinstance(keycode_raw, (int, float)):
            raise ValueError(f"events[{i}].keycode must be numeric.")
        keycode = int(keycode_raw)

        keysym = str(event.get("keysym", ""))
        char = str(event.get("char", ""))

        out_events.append(
            {
                "type": kind,
                "timestamp_ms": timestamp_ms,
                "keycode": keycode,
                "keysym": keysym,
                "char": char,
            }
        )

    if not any(ev["type"] == "keydown" for ev in out_events):
        raise ValueError("Raw run must include at least one keydown event.")

    return {
        "events": out_events,
    }


def build_feature_vector_from_raw_run(raw_run: Dict[str, Any]) -> np.ndarray:
    normalized = normalize_raw_run(raw_run)
    stream = RunCapture._collect_run_stream_from_events(normalized["events"])
    if stream is None:
        raise ValueError("Raw run has no usable keydown timing data.")
    chars, dwell, flight = stream
    return RunCapture._feature_vector_from_parts(chars, dwell, flight)
```

./keystroke_app/config.py

```
from pathlib import Path

APP_ROOT = Path(__file__).resolve().parent.parent

PROMPT_QUEUE_SIZE = 5
SENTENCE_DATASET_PATH = APP_ROOT / "medium_english_sentences.txt"
```

```
DEFAULT_DATASET_PATH = APP_ROOT / "keystroke_dataset.json"

DATASET_VERSION = 3
```

./keystroke_app/prompts.py

```
import random
from pathlib import Path
from typing import List, Sequence


def load_sentence_bank(path: Path) -> List[str]:
    sentences = [" ".join(line.strip().split()) for line in path.read_text(encoding="utf-8")
    sentences = [s for s in sentences if s]
    if len(sentences) < 1000:
        raise ValueError(f"Expected at least 1000 sentences in {path}, found {len(sentences)
    return sentences


class PromptQueue:
    def __init__(self, sentence_bank: Sequence[str], size: int):
        if size < 1:
            raise ValueError("Prompt queue size must be >= 1.")
        self._sentence_bank = list(sentence_bank)
        if not self._sentence_bank:
            raise ValueError("Sentence bank cannot be empty.")
        self._size = size
        self._queue: List[str] = []
        self.reset()

    def _random_sentence(self) -> str:
        return random.choice(self._sentence_bank)

    def reset(self):
        self._queue = [self._random_sentence() for _ in range(self._size)]

    def current(self) -> str:
        if not self._queue:
            self.reset()
        return self._queue[0]

    def advance(self):
        if not self._queue:
            self.reset()
            return
```

```
        self._queue.pop(0)
        self._queue.append(self._random_sentence())

    def as_text(self) -> str:
        return "\n".join(self._queue)
```

./keystroke_app/storage.py

```
from copy import deepcopy
import json
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import numpy as np

from .capture import RAW_RUN_FORMAT, build_feature_vector_from_raw_run, normalize_raw_run


@dataclass
class SessionData:
    enrollment_samples: List[np.ndarray]
    test_samples: List[np.ndarray]
    enrollment_raw_runs: List[Dict[str, Any]]
    test_raw_runs: List[Dict[str, Any]]
    reference: Optional[np.ndarray]
    score_threshold: float = 0.72


def _copy_samples(samples: List[np.ndarray]) -> List[np.ndarray]:
    return [np.array(s, dtype=np.float32, copy=True) for s in samples]


def _copy_raw_runs(raw_runs: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    return [deepcopy(run) for run in raw_runs]


def _serialize_raw_runs(raw_runs: List[Dict[str, Any]], field_name: str) -> List[Dict[str, A
    out: List[Dict[str, Any]] = []
    for i, run in enumerate(raw_runs):
        try:
            out.append(normalize_raw_run(run))
        except Exception as ex:
            raise ValueError(f"{field_name}[{i}] is invalid: {ex}") from ex
```

```
    return out


def _deserialize_raw_runs(raw: Any, field_name: str) -> Tuple[List[Dict[str, Any]], List[np.
    if raw is None:
        raise ValueError(f"'{field_name}' is required.")
    if not isinstance(raw, list):
        raise ValueError(f"'{field_name}' must be a list.")

    out_runs: List[Dict[str, Any]] = []
    out_features: List[np.ndarray] = []
    for i, item in enumerate(raw):
        try:
            run = normalize_raw_run(item)
            feat = build_feature_vector_from_raw_run(run)
        except Exception as ex:
            raise ValueError(f"{field_name}[{i}] is not a valid native key event run: {ex}")
        out_runs.append(run)
        out_features.append(np.asarray(feat, dtype=np.float32))
    return out_runs, out_features


def _deserialize_reference(raw: Any) -> Optional[np.ndarray]:
    if raw is None:
        return None
    arr = np.asarray(raw, dtype=np.float32)
    if arr.ndim != 1 or arr.size == 0:
        raise ValueError("'reference' must be a non-empty 1D numeric vector.")
    return arr


def _feature_dim(data: SessionData) -> Optional[int]:
    if data.reference is not None:
        return int(data.reference.shape[0])
    if data.enrollment_samples:
        return int(data.enrollment_samples[0].shape[0])
    if data.test_samples:
        return int(data.test_samples[0].shape[0])
    return None


def _validate_dimensions(data: SessionData):
    dim = _feature_dim(data)
    if dim is None:
        return
    if data.reference is not None and int(data.reference.shape[0]) != dim:
```

```
            raise ValueError("Reference vector dimension does not match dataset dimension.")
        for i, arr in enumerate(data.enrollment_samples):
            if int(arr.shape[0]) != dim:
                raise ValueError(f"Enrollment sample #{i + 1} has wrong feature dimension.")
        for i, arr in enumerate(data.test_samples):
            if int(arr.shape[0]) != dim:
                raise ValueError(f"Test sample #{i + 1} has wrong feature dimension.")
        for i, run in enumerate(data.enrollment_raw_runs):
            feat = build_feature_vector_from_raw_run(run)
            if int(feat.shape[0]) != dim:
                raise ValueError(f"Enrollment raw run #{i + 1} has wrong derived feature dimensi
        for i, run in enumerate(data.test_raw_runs):
            feat = build_feature_vector_from_raw_run(run)
            if int(feat.shape[0]) != dim:
                raise ValueError(f"Test raw run #{i + 1} has wrong derived feature dimension.")


    def build_reference_from_enrollment(enrollment_samples: List[np.ndarray]) -> Optional[np.nda
        if not enrollment_samples:
            return None
        X = np.stack(enrollment_samples, axis=0)
        return X.mean(axis=0).astype(np.float32)


    def session_data_to_payload(data: SessionData, dataset_version: int) -> Dict[str, Any]:
        _validate_dimensions(data)
        feature_dim = _feature_dim(data)
        enrollment_raw = _serialize_raw_runs(data.enrollment_raw_runs, "enrollment_raw_runs")
        test_raw = _serialize_raw_runs(data.test_raw_runs, "test_raw_runs")
        if len(data.enrollment_samples) != len(enrollment_raw):
            raise ValueError("Enrollment sample count must match enrollment raw run count.")
        if len(data.test_samples) != len(test_raw):
            raise ValueError("Test sample count must match test raw run count.")

        payload: Dict[str, Any] = {
            "type": "keystroke_dataset",
            "version": int(dataset_version),
            "created_unix": int(time.time()),
            "run_format": RAW_RUN_FORMAT,
            "score_threshold": float(data.score_threshold),
            "feature_dim": int(feature_dim) if feature_dim is not None else None,
            "num_enrollment_runs": int(len(data.enrollment_samples)),
            "num_test_runs": int(len(data.test_samples)),
            "num_enrollment_raw_runs": int(len(enrollment_raw)),
            "num_test_raw_runs": int(len(test_raw)),
            "enrollment_runs": enrollment_raw,
```

```
        "test_runs": test_raw,
        "reference": None if data.reference is None else data.reference.astype(float).tolist
    }
    return payload


def payload_to_session_data(payload: Dict[str, Any]) -> SessionData:
    run_format = payload.get("run_format")
    if run_format != RAW_RUN_FORMAT:
        raise ValueError(
            f"Unsupported run_format '{run_format}'. Expected '{RAW_RUN_FORMAT}'."
        )

    enrollment_runs_raw = payload.get("enrollment_runs")
    test_runs_raw = payload.get("test_runs")
    enrollment_raw_runs, enrollment_samples = _deserialize_raw_runs(enrollment_runs_raw, "en
    test_raw_runs, test_samples = _deserialize_raw_runs(test_runs_raw, "test_runs")

    reference = _deserialize_reference(payload.get("reference"))
    score_threshold = payload.get("score_threshold", 0.72)
    if not isinstance(score_threshold, (int, float)):
        raise ValueError("'score_threshold' must be numeric.")

    data = SessionData(
        enrollment_samples=enrollment_samples,
        test_samples=test_samples,
        enrollment_raw_runs=enrollment_raw_runs,
        test_raw_runs=test_raw_runs,
        reference=reference,
        score_threshold=float(score_threshold),
    )
    _validate_dimensions(data)

    feature_dim = payload.get("feature_dim")
    if isinstance(feature_dim, int):
        dim = _feature_dim(data)
        if dim is not None and dim != feature_dim:
            raise ValueError(f"Dataset feature_dim ({feature_dim}) does not match data vecto

    # If reference isn't stored but enrollment exists, derive it for immediate test use.
    if data.reference is None:
        data.reference = build_reference_from_enrollment(data.enrollment_samples)

    return data
```

```
def save_session_data(path: Path, data: SessionData, dataset_version: int):
    payload = session_data_to_payload(data, dataset_version)
    path.write_text(json.dumps(payload, indent=2), encoding="utf-8")


def load_session_data(path: Path) -> SessionData:
    payload = json.loads(path.read_text(encoding="utf-8"))
    return payload_to_session_data(payload)


def merge_session_data(base: SessionData, incoming: SessionData) -> SessionData:
    merged_enrollment = _copy_samples(base.enrollment_samples) + _copy_samples(incoming.enro
    merged_test = _copy_samples(base.test_samples) + _copy_samples(incoming.test_samples)
    merged_enrollment_raw = _copy_raw_runs(base.enrollment_raw_runs) + _copy_raw_runs(incomi
    merged_test_raw = _copy_raw_runs(base.test_raw_runs) + _copy_raw_runs(incoming.test_raw_

    merged_reference: Optional[np.ndarray]
    if merged_enrollment:
        merged_reference = build_reference_from_enrollment(merged_enrollment)
    elif incoming.reference is not None:
        merged_reference = np.array(incoming.reference, dtype=np.float32, copy=True)
    elif base.reference is not None:
        merged_reference = np.array(base.reference, dtype=np.float32, copy=True)
    else:
        merged_reference = None

    merged_threshold = float(incoming.score_threshold)
    merged = SessionData(
        enrollment_samples=merged_enrollment,
        test_samples=merged_test,
        enrollment_raw_runs=merged_enrollment_raw,
        test_raw_runs=merged_test_raw,
        reference=merged_reference,
        score_threshold=merged_threshold,
    )
    _validate_dimensions(merged)
    return merged


def merge_session_data_file(path: Path, incoming: SessionData, dataset_version: int) -> Sess
    if path.exists():
        base = load_session_data(path)
    else:
        base = SessionData(
            enrollment_samples=[],
            test_samples=[],
```

```
            enrollment_raw_runs=[],
            test_raw_runs=[],
            reference=None,
            score_threshold=0.72,
        )
    merged = merge_session_data(base, incoming)
    save_session_data(path, merged, dataset_version)
    return merged
```

./keystroke_app/verifier.py

```
from typing import Optional, Tuple

import numpy as np


class Verifier:
    def __init__(self):
        self.reference: Optional[np.ndarray] = None
        # Higher threshold is stricter. Tune this based on your data.
        self.score_threshold: float = 0.72

    def has_reference(self) -> bool:
        return self.reference is not None

    def clear(self):
        self.reference = None

    def fit(self, x: np.ndarray):
        self.reference = np.array(x, dtype=np.float32, copy=True)

    def score(self, X: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
        assert self.reference is not None
        ref = self.reference
        if X.ndim != 2 or X.shape[1] != ref.shape[0]:
            got_dim = X.shape[1] if X.ndim == 2 else "invalid"
            raise ValueError(f"Feature dimension mismatch: got {got_dim}, expected {ref.shap

        ref_norm = float(np.linalg.norm(ref)) + 1e-8
        x_norm = np.linalg.norm(X, axis=1) + 1e-8

        cosine = (X @ ref) / (x_norm * ref_norm)
        rel_l2 = np.linalg.norm(X - ref, axis=1) / ref_norm

        # Blend angular similarity and relative distance.
```

```
        scores = 0.7 * cosine + 0.3 * (1.0 - rel_l2)
        inlier = scores >= self.score_threshold
        return scores, inlier
```

./medium_english_sentences.txt

```
"A rolling stone gathers no moss" is a proverb.
"But don't you think that it's a little big?" asked the shopkeeper.
"He's a tiger when he's angry" is an example of metaphor.
"How long does it take to get to Vienna on foot?" he inquired.
"I can make it to my class on time," he thought.
"I can't bear to be doing nothing!" you often hear people say.
"I saw her five days ago," he said.
"I want that book," he said to himself.
"I'll be back in a minute," he added.
"I'm the happiest man in the world," Tom said to himself.
"Superman" is showing at the movie theater this month.
"Thank you, I'd love to have another piece of cake," said the shy young man.
"The good die young" is an old saying which may or may not be true.
"What should I do?" I said to myself.
"What will you have to do?" asked her friend.
1980 was the year that I was born.
67% of those who never smoked said they worried about the health effects of passive smoking.
A "renovator's dream" in real estate parlance generally means that the place is a real dump.
A 5% consumption tax is levied on purchases of most goods and services.
A 6% yield is guaranteed on the investment.
A baby has no knowledge of good and evil.
A baby is incapable of taking care of itself.
A bad cold has kept me from studying this week.
A bad cold prevented her from attending the class.
A bad habit, once formed, is difficult to get rid of.
A bad writer's prose is full of hackneyed phrases.
A bat flying in the sky looks like a butterfly.
A bat hunts food and eats at night, but sleeps during the day.
A bat is no more a bird than a rat is.
A bead of sweat started forming on his brow.
A beam of sunlight came through the clouds.
A bear will not touch a dead body.
A beautiful lake lies just beyond the forest.
A beautiful salesgirl waited on me in the shop.
A beautiful woman was seated one row in front of me.
A belt keeps your pants from falling down.
A bicycle will rust if you leave it in the rain.
A big bomb fell, and a great many people lost their lives.
A big bridge was built over the river.
```

A big surprise was waiting for me at home.
A big wave swept the man off the boat.
A bird can glide through the air without moving its wings.
A bird in the hand is better than two in the bush.
A bird in the hand is worth two in the bush.
A bird is known by its song and a man by his way of talking.
A bird was flying high up in the sky.
A blast of cold air swept through the house.
A blender lets you mix different foods together.
A blind person's hearing is often very acute.
A boat suddenly appeared out of the mist.
A book can be compared to a friend.
A book not worth reading is not worth buying in the first place.
A book without preface is like a body without a soul.
A bookstore in that location wouldn't make enough money to survive.
A bottle of shampoo costs as much as a tube of toothpaste.
A boy like Tom doesn't deserve a girl like Mary.
A boy needs a father he can look up to.
A boy of seventeen is often as tall as his father.
A boy snatched my purse as he rode by on his bicycle.
A brass band is marching along the street.
A bright red ladybug landed on my fingertip.
A bunch of people died in the explosion.
A bunch of people told me not to eat there.
A bunch of people were standing outside waiting.
A burglar broke into my house while I was away on a trip.
A burglar broke into the bank last night.
A burglar made away with my wife's diamond ring.
A bus transported us from the airport to the city.
A business cycle is a recurring succession of periods of prosperity and periods of depressio
A bust of Aristotle stands on a pedestal in the entryway.
A bystander videotaped the police beating using their cell phone.
A caged cricket eats just as much as a free cricket.
A camel is a horse designed by a committee.
A capital letter is used at the beginning of a sentence.
A captain controls his ship and its crew.
A captain is in charge of his ship and its crew.
A car drew up at the main gate.
A car drew up in front of my house.
A car in the parking lot is on fire.
A car is a must for life in the suburbs.
A caravan of fifty camels slowly made its way through the desert.
A careful reader would have noticed the mistake.
A careless person is apt to make mistakes.
A cargo vessel, bound for Athens, sank in the Mediterranean without a trace.
A castle stands a little way up the hill.

A cat came out from under the desk.
A cat can see much better at night.
A cat has a tail and four legs.
A cat was sleeping in the bass drum.
A chain is made up of many links.
A chain is no stronger than its weakest link.
A chain is only as strong as its weakest link.
A chain of events led to the outbreak of the war.
A chance like this only comes along once in a blue moon.
A change of air will do you a lot of good.
A cheetah can run as fast as 70 miles per hour.
A child whose parents are dead is called an orphan.
A clear conscience is the sure sign of a bad memory.
A cold wind was blowing on his face.
A combination of several mistakes led to the accident.
A common way to finance a budget deficit is to issue bonds.
A company that stifles innovation can't hope to grow very much.
A comparable car would cost far more in Japan.
A conservative tie is preferable to a loud one for a job interview.
A considerable amount of money was appropriated for the national defense.
A contract with that company is worth next to nothing.
A couple of flights were delayed on account of a minor accident.
A cowboy is driving cattle to the pasture.
A crowd of people gathered in the street.
A crowd of people gathered to see the parade.
A crowd soon gathered around the fire engine.
A crystal chandelier was hanging over the table.
A cup of coffee cost 200 yen in those days.
A customs official asked me to open my suitcase.
A dance will be held in the school auditorium this Friday evening from 7:30 to 10:00.
A day without laughter is a day wasted.
A debugger is a program which allows you to find errors in source code.
A detective arrived upon the scene of the crime.
A dictionary is an important aid in language learning.
A dish can be spicy without being hot.
A doctor should never let a patient die.
A doctor told me that eating eggs was bad for me.
A doctor tried to remove the bullet from his back.
A doctor tried to remove the bullet from the president's head.
A doctor's instruments must be kept absolutely clean.
A dog can run faster than a man can.
A dog has a sharp sense of smell.
A dog jumped onto the chair and lay motionless for five minutes.
A dog that barks all the time doesn't make a good watch dog.
A dog was run over by a truck.
A dog's sense of smell is much keener than a human's.

```
A dollar is equal to a hundred cents.
A dolphin is no more a fish than a dog is.
A dreary landscape spread out for miles in all directions.
A driver's job is not as easy as it looks.
A drunk driver was responsible for the car accident.
A drunk man's words are a sober man's thoughts.
A drunkard is somebody you don't like and who drinks as much as you do.
A drunken man was sleeping on the bench.
A dry spell accounts for the poor crop.
A dye was injected into a vein of the patient's arm.
A fall from that height would be fatal.
A fallen leaf floated on the surface of the water.
A fat man seldom dislikes anybody very hard or for very long.
A fat white cat sat on a wall and watched them with sleepy eyes.
A female friend of mine loves to go to gay bars with me.
A fence separates the garden from the path.
A few customers have just walked into the store.
A few days ago, you didn't even want to talk to me.
A few days later, Tom found another job.
```

./test.py

```
from keystroke_app.app import main


if __name__ == "__main__":
    main()
```